

Readiness Probe i Liveness Probe

Dlaczego Pod może być Running, a aplikacja nadal nie działa?

```
kubectl describe pod <pod>  
kubectl get endpoints
```



Cel modułu

zrozumieć różnicę między Running, Ready i zdrową aplikacją
zobaczyć wpływ readinessProbe na Service i endpoints
zobaczyć wpływ livenessProbe na restarty kontenera
nauczyć się diagnozować probe przez describe, logs i events
przećwiczyć naprawę błędnych probe w manifeście



Running $\neq$ Ready

Running
kontener działa

Ready
aplikacja gotowa

Service
może wysłać ruch

Dla QA ważne: Pod może istnieć i działać, ale nie musi jeszcze obsługiwać ruchu.



Czym są probe?

probe to cykliczny test wykonywany przez kubelet
Kubernetes nie zna aplikacji — zna tylko wynik testu
test może być HTTP, TCP albo exec

readinessProbe

czy aplikacja może przyjmować ruch?

livenessProbe

czy kontener należy zrestartować?



readinessProbe

```
readinessProbe:  
  httpGet:  
    path: /  
    port: 80  
  initialDelaySeconds: 5  
  periodSeconds: 5
```

Jeśli readinessProbe nie przechodzi, Pod nie trafia do endpoints Service.



Lab: błędny readinessProbe

```
kubectl apply -f readiness-broken.yaml  
kubectl get pods  
kubectl get endpoints
```

```
readinessProbe:  
  httpGet:  
    path: /healthz  
    port: 80
```

Nginx nie ma domyślnie endpointu /healthz, więc Pod nie będzie Ready.



Diagnoza readiness

```
kubectl describe pod <pod-name>  
kubectl get events --sort-  
by=.metadata.creationTimestamp  
kubectl get endpoints  
kubectl describe svc <service-name>
```

Sygnal ostrzegawczy: Pod jest Running, ale nie ma go w endpoints Service.



Naprawa readiness

```
kubectl edit deployment readiness-demo  
# w vim poprawiamy path  
:wq
```

Było

```
path: /healthz
```

Ma być

```
path: /
```

```
watch -n2 "kubectl get pods -o wide"
```



Service i endpoints



```
kubectl get endpoints  
kubectl describe svc <service-name>
```

Brak endpointów bardzo często oznacza problem z readiness albo selectorami Service.



livenessProbe

```
livenessProbe:  
  httpGet:  
    path: /  
    port: 80  
  initialDelaySeconds: 10  
  periodSeconds: 5
```

Jeśli livenessProbe nie przechodzi, Kubernetes restartuje kontener.



Lab: błędny livenessProbe

```
kubectl apply -f liveness-broken.yaml  
kubectl get pods  
kubectl describe pod <pod-name>
```

```
livenessProbe:  
  httpGet:  
    path: /not-existing  
    port: 80
```

Efekt: rosnący RESTARTS i eventy o failed liveness probe.



Naprawa liveness

```
kubectl edit deployment liveness-demo  
# w vim poprawiamy path  
:wq
```

Było

```
path: /not-existing
```

Ma być

```
path: /
```

```
watch -n2 "kubectl get pods -o wide"
```



Typy probe

httpGet

HTTP endpoint
np. /health

tcpSocket

czy port
odpowiada

exec

komenda
w kontenerze

W aplikacjach webowych najczęściej spotkasz httpGet.



Najważniejsze parametry

```
initialDelaySeconds: 10    # kiedy zacząć testować  
periodSeconds: 5          # jak często testować  
timeoutSeconds: 2         # ile czekać na odpowiedź  
failureThreshold: 3       # ile porażek oznacza problem  
successThreshold: 1       # ile sukcesów oznacza OK
```

Zbyt agresywne probe potrafią same spowodować niestabilność aplikacji.



startupProbe — warto znać

startupProbe chroni wolno startujące aplikacje przed zbyt szybkim restartem przez livenessProbe.

```
startupProbe:  
  httpGet:  
    path: /startup  
    port: 8080  
  failureThreshold: 30  
  periodSeconds: 10
```



Workflow diagnostyczny

```
kubectl get pods  
kubectl describe pod <pod-name>  
kubectl get endpoints  
kubectl describe svc <service-name>  
kubectl logs <pod-name>  
kubectl get events --sort-by=.metadata.creationTimestamp  
watch -n2 "kubectl get pods -o wide"
```

Przy probie najważniejsze są: describe, events, endpoints i licznik restartów.



Mini lab: przebieg

uruchamiamy Deployment z błędnym readinessProbe
sprawdzamy, że Pod nie trafia do endpoints
naprawiamy probe przez `kubectl edit deployment`
uruchamiamy Deployment z błędnym livenessProbe
obserwujemy restarty i naprawiamy path

```
kubectl edit deployment <deployment-name>
```



Do zapamiętania

readiness

decyduje, czy Pod dostaje ruch z Service

liveness

decyduje, czy kontener zostanie zrestartowany

Running to nie zawsze Ready. Ready to dopiero gotowość do obsługi ruchu.

